

ID del Caso: CP-001

Nivel y Enfoque: Sistema / Caja Negra

Combinación: Tolerancia a fallos + Análisis de Valores Límite

Objetivo: DESTRUCTIVO. Violar el límite de la regla de negocio creando el tercer préstamo activo en simultáneo para corromper el validador.

Precondiciones: El usuario existe en BD y tiene exactamente 2 préstamos activos (límite máximo).

Datos de Entrada: ``correoUsuario` = "tester@ucab.edu", `isbn` = 9781234567897, `fechaPrestamoStr` = "2026-07-12"`

Pasos de Ejecución:

1. Autenticarse con rol USUARIO y obtener token.
2. Interceptar la petición de red.
3. Enviar un POST a ``/api/prestar`` con el payload de entrada.
4. Repetir la misma petición POST en menos de 50ms usando hilos concurrentes para provocar una condición de carrera.

Resultados Esperados: El sistema DEBE rechazar la petición con HTTP 400 y el mensaje "El usuario ya tiene 2 préstamos activos." SIN crear el tercer préstamo ni generar un error 500.

Criterios de Aceptación: El registro en BD muestra ``countByUsuarioldAndFechaDevolucionIsNull` = 2.`

Trazabilidad: ``PrestamoService.crearPrestamo()`` (líneas 36-37) y ``Controller.registrarPrestamo()``.

ID del Caso: CP-002

Nivel y Enfoque: Unitarias / Caja Blanca

Combinación: Madurez + Cobertura de Ramas / Decisiones

Objetivo: DESTRUCTIVO. Forzar la evaluación de la rama donde un usuario con amonestaciones intenta engañar al sistema para renovar un libro.

Precondiciones: BD en memoria (H2) instanciada. Un préstamo con estado "finalizado". Una amonestación asociada al usuario en estado `verificada = false`.

Datos de Entrada: `prestamold` = 1, `usuarioRol` = "BIBLIOTECARIO".

Pasos de Ejecución:

1. Configurar los mocks de `amonestacionRepository` para que `existsByUsuarioldAndVerificadaFalse` retorne `true`.
2. Ejecutar `prestamoService.renovarPrestamo(1, "BIBLIOTECARIO")` en JUnit.

Resultados Esperados: La función retorna exactamente "No se puede renovar el préstamo. El usuario tiene amonestaciones pendientes de verificación." y la rama se cubre al 100%.

Criterios de Aceptación: El `prestamoRepository.save()` NUNCA debe ser invocado (`verify(prestamoRepository, never()).save(any())`).

Trazabilidad: `PrestamoService.renovarPrestamo()` (líneas 137-140).

ID del Caso: CP-003

Nivel y Enfoque: Sistema / Caja Negra

Combinación: Disponibilidad + Partición de Equivalencia

Objetivo: DESTRUCTIVO. Inyectar datos en una partición inválida anómala para tumbar el endpoint y afectar la disponibilidad del servicio.

Precondiciones: API desplegada y accesible.

Datos de Entrada: `correoUsuario` = NULL, `isbn` = -1, `fechaPrestamoStr` = "9999-99-99" (Formato de fecha inválido).

Pasos de Ejecución:

1. Construir un JSON con el `isbn` negativo y la fecha irreal.
2. Enviar petición POST a `/api/prestar` sin cabecera de autenticación.

Resultados Esperados: El sistema debe manejar la excepción del parseo de fecha (`LocalDate.parse()`) devolviendo un error HTTP 400 controlado. NO DEBE arrojar un 500 Internal Server Error que mate el hilo del servidor Tomcat.

Criterios de Aceptación: El servicio sigue respondiendo a peticiones de salud (health check) tras el ataque.

Trazabilidad: `PrestamoService.crearPrestamo()` (línea 47).

ID del Caso: CP-004

Nivel y Enfoque: Integración / Caja Gris

Combinación: Tolerancia a fallos + Adivinación de Errores (Error Guessing)

Objetivo: DESTRUCTIVO. Explotar la lógica de devolución para incrementar falsamente el inventario de libros (Ataque de Doble Gasto).

Precondiciones: Préstamo ID 5 activo. Libro ID 10 con `cantidad` = 5.

Datos de Entrada: `prestamold` = 5.

Pasos de Ejecución:

1. Abrir dos terminales paralelas (o usar JMeter).
2. Ejecutar simultáneamente `POST /api/prestamos/devolver?prestamold=5` en ambas terminales.

Resultados Esperados: La primera petición retorna "Préstamo devuelto con éxito." (cantidad a 6). La segunda DEBE retornar "El préstamo ya fue devuelto." y NO aumentar la cantidad a 7.

Criterios de Aceptación: Al consultar la BD, el libro ID 10 tiene `cantidad` = 6, y la BD no tiene registros inconsistentes.

Trazabilidad: `PrestamoService.devolverPrestamo()` (líneas 72-74 y 81-83).

ID del Caso: CP-005

Nivel y Enfoque: Integración / Caja Blanca

Combinación: Capacidad de Recuperación + Análisis de Flujo de Datos

Objetivo: DESTRUCTIVO. Romper el flujo de la transacción a la mitad para validar si el sistema deja datos corruptos o huérfanos.

Precondiciones: Contexto de Spring cargado. `prestamoRepository` funciona, pero se inyecta un mock en `libroRepository` forzado a arrojar `DataAccessException`.

Datos de Entrada: Petición válida para crear préstamo.

Pasos de Ejecución:

1. Llamar a `PrestamoService.crearPrestamo`.
2. La línea 58 (`prestamoRepository.save`) guarda el préstamo en BD.
3. La línea 61 (`libroRepository.save`) arroja la excepción programada.

Resultados Esperados: La prueba VA A FALLAR exponiendo el defecto arquitectónico. Como el método no tiene la anotación `@Transactional`, el sistema no tiene capacidad de recuperación (rollback). El préstamo queda guardado, pero el libro no se descuenta.

Criterios de Aceptación: El tester reporta el hallazgo crítico (INC) exigiendo refactorización y control transaccional inmediato.

Trazabilidad: `PrestamoService.crearPrestamo()` (líneas 58 a 61).

ID del Caso: CP-006

Nivel y Enfoque: Integración / Caja Negra

Combinación: Tolerancia a fallos + Transición de Estados (State Transition Testing)

Objetivo: DESTRUCTIVO. Forzar una transición de estado ilegal en la máquina de estados del préstamo.

Precondiciones: Préstamo ID 8 en estado "activo" (aún no devuelto). Autenticado como "BIBLIOTECARIO".

Datos de Entrada: `id` = 8.

Pasos de Ejecución:

1. Validar en BD que el préstamo está activo (`fechaDevolucion` es NULL).
2. Enviar petición PUT directa a `/api/prestamos/8/renovar`.

Resultados Esperados: El sistema debe impedir la transición del estado "activo" al estado "renovado" saltándose el estado intermedio "finalizado". Debe retornar HTTP 400 con "Solo se pueden renovar préstamos finalizados."

Criterios de Aceptación: El préstamo mantiene su `fechaLimite` original y su estado no se altera.

Trazabilidad: `PrestamoService.renovarPrestamo()` (líneas 132-134).

ID del Caso: CP-007

Nivel y Enfoque: Unitarias / Caja Blanca

Combinación: Madurez + Cobertura de Instrucciones (Statement Testing)

Objetivo: DESTRUCTIVO. Forzar la ejecución de las instrucciones ocultas de penalización simulando un viaje en el tiempo.

Precondiciones: Mock del repositorio de préstamos. Préstamo vencido hace 30 días (``fechaLimite` es `hoy.minusDays(30)``).

Datos de Entrada: ``prestamold` = 99`.

Pasos de Ejecución:

1. Llamar a ``prestamoService.devolverPrestamo(99)``.
2. Validar con un capturador de argumentos (``ArgumentCaptor`` de Mockito) qué objeto se mandó a guardar.

Resultados Esperados: Todas las instrucciones de penalización (líneas 86-97) DEBEN ejecutarse. El capturador comprueba que la amonestación se creó con ``monto = 100.0``.

Criterios de Aceptación: 100% de cobertura de código en el bloque ``if (hoy.isAfter(prestamo.getFechaLimite()))``.

Trazabilidad: ``PrestamoService.devolverPrestamo()`` (líneas 86-97).

ID del Caso: CP-008

Nivel y Enfoque: Aceptación / Caja Negra

Combinación: Capacidad de Recuperación + Pruebas Aleatorias (Random Testing / Fuzzing)

Objetivo: DESTRUCTIVO. Ahogar el parseo JSON del backend inyectando payloads gigantescos.

Precondiciones: API ejecutándose en entorno Beta.

Datos de Entrada: Petición con cuerpo JSON mutado: `{ "texto": "A" \* 10000000 (10MB de basura) `

Pasos de Ejecución:

1. Generar payload de 10MB.
2. Autenticarse como usuario normal.
3. Enviar PUT a `/api/resenas/1` con el payload destructivo.

Resultados Esperados: El servidor web debe interceptar la petición por exceso de tamaño (`Payload Too Large`) recuperándose sin colapsar el servicio para los demás usuarios. NO debe guardar la reseña.

Criterios de Aceptación: Retorna HTTP 413, los logs advierten el rechazo, y el consumo de RAM del servidor se normaliza inmediatamente.

Trazabilidad: `Controller.editarResena()` (línea 258).

ID del Caso: CP-009

Nivel y Enfoque: Integración / Caja Gris

Combinación: Disponibilidad + Pruebas de Tablas de Decisión

Objetivo: DESTRUCTIVO. Combinar lógicas de autorización ilógicas para lograr manipulación cruzada de datos.

Precondiciones: Usuario A autenticado. Comentario ID 5 pertenece al Usuario B.

Datos de Entrada: `id` = 5, Bearer Token del Usuario A.

Pasos de Ejecución:

1. Mapeo en tabla de decisión: ¿Autenticado? (V). ¿Existe comentario? (V). ¿Es dueño del comentario? (F).

2. Enviar DELETE a `/api/comentarios-resena/5` usando el token del Usuario A.

Resultados Esperados: Según la tabla de decisión de seguridad, la combinación `V, V, F` DEBE ser interceptada en la línea 344 y retornar HTTP 403 Forbidden.

Criterios de Aceptación: El comentario ID 5 sigue intacto en BD asociado al Usuario B.

Trazabilidad: `Controller.eliminarComentarioResena()` (líneas 344-346).

ID del Caso: CP-010

Nivel y Enfoque: Sistema / Caja Gris

Combinación: Tolerancia a fallos + Pruebas de Escenario (Scenario / Use Case Testing)

Objetivo: DESTRUCTIVO. Eliminar un perfil de usuario con dependencias duras activas (préstamos no devueltos y deudas) para corromper la integridad referencial de la base de datos.

Precondiciones: Usuario "MOROSO" con 1 préstamo sin devolver y 1 amonestación activa.

Datos de Entrada: Petición DELETE autenticada por "MOROSO".

Pasos de Ejecución:

1. Iniciar sesión como "MOROSO".
2. Enviar DELETE a `/api/usuarios`.
3. Intentar consultar el préstamo o el libro asociado en la BD.

Resultados Esperados: La prueba va a exponer OTRA FALLA ARQUITECTÓNICA. En el `Controller` (línea 186) se llama a `usuarioService.eliminarUsuario(correo)` de forma directa. Si la base de datos es relacional y no tiene un manejo en cascada configurado correctamente (o validaciones previas), el sistema arrojará un error 500 por violación de **foreign key constraint**, o peor aún, dejará el libro secuestrado y el inventario corrompido de forma irreversible.

Criterios de Aceptación: La prueba expone el defecto. El tester reporta el hallazgo documentando la falta de validación antes del DELETE.

Trazabilidad: `Controller.eliminarPerfil()` (línea 182-188).

ID del Caso: CP-011 (Corregido)

Nivel y Enfoque: Integración / Caja Gris

Combinación: Tolerancia a fallos + Adivinación de Errores

Objetivo: Someter al sistema a un fallo de dependencia (lentitud extrema en la BD) para verificar si el hilo de ejecución de Tomcat se bloquea o si el sistema corta por lo sano.

Precondiciones: amonestacionRepository mockeado (o base de datos detrás de un proxy como Toxiproxy) configurado para inyectar una latencia de 30 segundos. Datos de

Entrada: Petición POST válida a /api/prestamos/devolver que dispara una amonestación.

Pasos de Ejecución:

1. Configurar la dependencia (BD o Mock) para que demore 30 segundos en responder al save().
2. Ejecutar la petición HTTP.

Resultados Esperados: El sistema DEBE aplicar un timeout a nivel de consulta (ej. configurando query.timeout en JPA) para no dejar el hilo bloqueado 30 segundos, lanzando una excepción controlada rápidamente.

Criterios de Aceptación: La API aborta y responde en < 500ms. El hilo del servidor no queda secuestrado.

ID del Caso: CP-012

Nivel y Enfoque: Integración / Caja Blanca

Combinación: Capacidad de Recuperación + Pruebas de Recuperación (Disaster Recovery Testing)

Objetivo: DESTRUCTIVO. Asesinar el motor de base de datos en plena transacción concurrente para validar el manejo seguro de excepciones y el restablecimiento autónomo del *pool* de conexiones.

Precondiciones: Aplicación web levantada y conectada a la base de datos.

Datos de Entrada: Peticiones concurrentes GET a `/api/libros`.

Pasos de Ejecución:

1. Lanzar un script de JMeter bombardeando el endpoint del catálogo de libros con 20 hilos concurrentes.
2. Durante el pico de tráfico, detener forzosamente el servicio del motor de base de datos (ej. `sudo systemctl stop postgres` o apagando abruptamente el contenedor Docker).
3. Evaluar los *response bodies* y *headers* HTTP recibidos por los clientes.
4. Reiniciar la base de datos tras 15 segundos y volver a inyectar carga.

Resultados Esperados: Durante la caída, el sistema debe interceptar la `DataAccessException` y retornar un mensaje HTTP 503 o 500 genérico controlado, ****OCULTANDO POR COMPLETO**** el stacktrace y las credenciales JDBC. Al reiniciar la BD, el *Connection Pool* (ej. HikariCP de Spring) debe reconectarse automáticamente y procesar peticiones nuevas SIN necesidad de reiniciar el servidor web Tomcat.

Criterios de Aceptación: Ningún stacktrace expuesto en el response JSON. El sistema retoma la conexión a la BD de forma autónoma.

Trazabilidad: Manejo global de Excepciones (`@ControllerAdvice`), Configuración del Pool de Conexiones en `application.properties` y `Controller.obtenerLibros()`.

ID del Caso: CP-013

Nivel y Enfoque: Sistema / Caja Negra

Combinación: Madurez + Pruebas de Resistencia (Soak Testing)

Objetivo: DESTRUCTIVO. Provocar un *Memory Leak* (Fuga de Memoria) o saturación extrema del Recolector de Basura (Garbage Collector) manteniendo la aplicación bajo estrés constante durante 24 horas.

Precondiciones: API monitoreada en staging con JMX, VisualVM o una herramienta APM (DataDog/New Relic) para graficar el *Heap Memory* de la JVM.

Datos de Entrada: Flujo repetitivo: Login -> Prestar -> Generar Amonestación -> Pagar.

Pasos de Ejecución:

1. Iniciar JMeter con 50 usuarios virtuales ejecutando iteraciones del flujo descrito sin pausas (Delay = 0).
2. Mantener la prueba activa durante 24 horas ininterrumpidas.
3. Monitorear el comportamiento de retención de objetos en la memoria *Heap* y la CPU del servidor.

Resultados Esperados: El consumo de memoria debe mostrar un patrón de "sierra" estable, evidenciando que el Garbage Collector limpia objetos con éxito. El sistema NO DEBE acumular objetos huérfanos (ej. retención indebida en el cálculo masivo de las penalizaciones o almacenamiento infinito de sesiones en memoria) que eventualmente disparen un colapso por `java.lang.OutOfMemoryError`.

Criterios de Aceptación: Cero excepciones OOM en logs. El percentil 95 del tiempo de respuesta se degrada menos del 10% respecto al inicio de la prueba tras 24 horas de actividad.

Trazabilidad: Arquitectura completa del `Controller`, `PrestamoService`, `AmonestacionService` y la persistencia del ORM.

ID del Caso: CP-014

Nivel y Enfoque: Sistema / Caja Blanca

Combinación: Tolerancia a fallos + Análisis de Valores Límite

Objetivo: DESTRUCTIVO. Forzar un cálculo injusto en las penalizaciones de los préstamos aprovechando la debilidad en el manejo de zonas horarias y el cruce del milisegundo de la medianoche.

Precondiciones: Un préstamo activo con ``fechaLimite` = "2026-07-15"` (Formato que no contiene hora). El servidor está en zona horaria UTC, y el tester/cliente se encuentra en UTC-4.

Datos de Entrada: Petición POST a ``/api/prestamos/devolver`` en el límite exacto del cambio de día.

Pasos de Ejecución:

1. Modificar artificialmente el reloj del entorno para emitir la petición de devolución desde el cliente exactamente a las 23:59:59 del 15 de julio (hora local).
2. Retener el paquete HTTP y liberarlo para que la línea de ejecución alcance el servidor en el milisegundo "00:00:01" del 16 de julio (hora del servidor UTC).
3. Validar si se genera o no una amonestación en BD.

Resultados Esperados: ¡Exponer el defecto arquitectónico! Al utilizar ``LocalDate.now()`` en la línea 76 de tu ``PrestamoService``, el software ignora el ``ZoneId`` y la hora exacta (precisión que aportaría ``LocalDateTime`` o ``Instant``). El servidor creará equivocadamente que el usuario devolvió el libro tarde debido a los milisegundos de latencia en red y el cruce asíncrono de medianoche, clavándole una amonestación injustificada de 100\$.

Criterios de Aceptación: La prueba demuestra el fallo en la validación del límite horario. Se genera la Incidencia Técnica (INC) para exigir una refactorización de la lógica de fechas utilizando estándares ISO-8601 con UTC (``Instant.now()``).

Trazabilidad: ``PrestamoService.devolverPrestamo()`` (líneas 76 y 86-87).